

EXERCISE 1 (1 + 1.5 + 1.5 + 1.5 + 1.5 = 7 POINTS).

The "pgm1" (written on page 1 of the answer sheet) allows to decompose a number into the product of its prime factors, then use this decomposition to calculate the GCD (Greatest Common Divisor) and LCM (Least Common Multiple) of two numbers. Complete the "pgm1" by writing in algorithmic language the following modules:

1. The "**Prime**" function that determines whether a number is prime or not. Recall that a prime number has no divisors other than 1 and itself.
2. The "**InitArray**" procedure that fills an array (of size 200) with the first 100 prime numbers in even-indexed cells. Odd-indexed cells are initialized to zero.

Index	0	1	2	3	4	5	6	7	8	9	10	11	12	13		199
Cell	2	0	3	0	5	0	7	0	11	0	13	0	17	0		0

3. The "**Decomp_prime**" procedure that decomposes an integer into the product of its prime factors and returns the list of factors and their powers in the array T as follows:
 - $a = 360 = 2 * 2 * 2 * 3 * 3 * 5 = (2 \uparrow 3) * (3 \uparrow 2) * (5 \uparrow 1) \rightarrow Ta = (2, 3, 3, 2, 5, 1, 7, 0, 11, 0, 13, 0, \dots)$
 - $b = 9702 = 2 * 3 * 3 * 7 * 7 * 11 = (2 \uparrow 1) * (3 \uparrow 2) * (7 \uparrow 2) * (11 \uparrow 1) \rightarrow Ta = (2, 1, 3, 2, 5, 0, 7, 2, 11, 1, ..)$
 - The odd-indexed cell $(2i+1)$ contains the power of the prime number in the even-indexed cell $(2i)$.
4. The "**GCD_decomp**" function that calculates the GCD of two numbers a and b based on their decompositions Ta and Tb. Recall that the GCD is the product of common prime factors raised to the minimum power.
 - $GCD_decomp(Ta, Tb) = (2 \uparrow 1) * (3 \uparrow 2) = 18.$
5. The "**LCM_decomp**" function that calculates the LCM of numbers a and b based on their decompositions Ta and Tb. Recall that the LCM is the product of common prime factors raised to the maximum power, and the non-common prime factors.
 - $LCM_decomp(Ta, Tb) = (2 \uparrow 3) * (3 \uparrow 2) * (5 \uparrow 1) * (7 \uparrow 2) * (11 \uparrow 1) = 194040$

EXERCISE 2 (1 + 2 + 2 = 5 POINTS).

The "code2" (written on page 2 of the answer sheet) allows to:

- a. Initialize an array of 4 digit numbers.
- b. Sort this array using RADIX sort.
- c. Interclassify the array in three parts: the Biprime numbers at the beginning of the array, the even numbers at the end and the rest in the middle.
- d. Display the final array.

Complete the "code2" by writing in C language the following modules:

1. The "**Biprime**" function that determines if a number is Biprime. A Biprime N is a 4 digit number composed of two 2 digit numbers ($N=N_1N_2$, $N_1 \geq 10$ and $N_2 \geq 10$) which are both prime. Examples: $N=2313$ ($N_1=23$, $N_2=13$) and $N=6131$ ($N_1=61$, $N_2=31$)
2. The "**Interclassify**" procedure that places the Biprime numbers from the array at the beginning, then the even numbers at the end and the rest in the middle.

3. The "RADIX" procedure that sorts an array of 4 digit numbers using radix sort. Radix sort allows to organize (sort) integers by examining their digits from right to left. For an array of 4 digit numbers, the algorithm proceeds in 4 passes, one for each position (units, tens, hundreds, thousands). At each pass, it sorts the array in ascending order based on the selection sort principle: For each digit from 0 to 9, apply selection sort considering the minimum as the current digit. Thus, place the zeros (if they exist) in the first cells, then the ones, and so on.

Here is an example of the "code2" trace :

Index	a. Read	b. Radix sort				c. Interclassify		d. Display
		1 st Pass	2 nd Pass	3 rd Pass	4 th Pass	1 st Pass	2 nd Pass	
0	2313	102 0	23 13	10 20	10 20	2313	2313	2313
1	8345	613 1	10 20	6 13 1	2313	6131	6131	6131
2	1020	453 2	45 32	2 313	2348	2348	8763	8763
3	6131	231 3	61 31	8 345	4532	4532	8345	8345
4	4532	876 3	83 45	2 348	6131	1020	1020	1020
5	8763	834 5	23 48	4 532	8345	8345	4532	4532
6	2348	234 8	87 63	8 763	8763	8763	2348	2348

EXERCISE 3 (1 + 1 + 2 = 4 POINTS).

Complete the functions of the "code3" (written on page 4 of the answer sheet) which allows reading a square matrix of order n and checking the following properties:

1. Diagonal: if all its elements outside the main diagonal are zero.
2. Symmetric: if it is equal to its transpose.
3. Magic: if the sum of the numbers in each row, column and diagonal is equal to the same constant, called the magic constant (in the example below, mag_const = 34).

Example of 4x4 matrices	Diagonal	Symmetric	Magic
	$\begin{pmatrix} 9 & 0 & 0 & 0 \\ 0 & 8 & 0 & 0 \\ 0 & 0 & 5 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$	$\begin{pmatrix} 1 & 3 & 5 & 7 \\ 3 & 2 & 4 & 6 \\ 5 & 4 & 8 & 0 \\ 7 & 6 & 0 & 9 \end{pmatrix}$	$\begin{pmatrix} 1 & 14 & 15 & 4 \\ 12 & 7 & 6 & 9 \\ 8 & 11 & 10 & 5 \\ 13 & 2 & 3 & 16 \end{pmatrix}$

EXERCISE 4 (2 + 1 + 1 = 4 POINTS).

We want to calculate the semester average of a university student. To do this, we need his information illustrated in Figure (drawn on page 5 of the answer sheet). Using algorithmic language,

1. create a data structure to store the student's information.
2. write the function "avg_module" to calculate the average of a module according to the value of the TCA variable which determines the type of Continuous Assessment (CA) as shown in the following table:

TCA	0	1	2	3
CA grade	CA = 0	CA = TD	CA = TP	CA = (TD + TP)/2
Average	100% Exam	60% Exam + 40% CA		

3. write the function "avg_semester" to calculate the semester average.

EXERCICE 1 (1 + 1.5 + 1.5 + 1.5 + 1.5 = 7 POINTS).

Le « pgm1 » (écrit sur la page 1 de feuille de réponse) permet de décomposer un nombre en produit de ses facteurs premiers, puis utiliser cette décomposition pour calculer le plus grand commun diviseur (GCD, Greatest Common Divisor) et le plus petit commun multiple (LCM, Least Common Multiple) de deux nombres. Compléter le « pgm1 » en écrivant en langage algorithmique les modules suivants :

1. La fonction « **Prime** » qui détermine si un nombre est premier ou non. Rappelons qu'un nombre premier n'accepte aucun diviseur excepté 1 et lui-même.
2. La procédure « **InitArray** » qui remplit un tableau (de taille 200) avec les 100 nombres premiers dans les cases d'indice pair. Les cases d'indice impair sont initialisées à zéros.

Indice	0	1	2	3	4	5	6	7	8	9	10	11	12	13		199
Case	2	0	3	0	5	0	7	0	11	0	13	0	17	0		0

3. La procédure « **Decomp_prime** » qui décompose un nombre en produit de ses facteurs premiers et retourne la liste des facteurs et leur puissance dans le tableau T comme suit :
 - $a = 360 = 2 * 2 * 2 * 3 * 3 * 5 = (2 \uparrow 3) * (3 \uparrow 2) * (5 \uparrow 1) \rightarrow Ta = (2, 3, 3, 2, 5, 1, 7, 0, 11, 0, 13, 0, \dots)$
 - $b = 9702 = 2 * 3 * 3 * 7 * 7 * 11 = (2 \uparrow 1) * (3 \uparrow 2) * (7 \uparrow 2) * (11 \uparrow 1) \rightarrow Ta = (2, 1, 3, 2, 5, 0, 7, 2, 11, 1, ..)$
 - La case d'indice impair ($2*i+1$) contient la puissance du nombre premier de la case d'indice pair ($2*i$).
4. La fonction « **GCD_decomp** » qui calcule le GCD de deux nombres a et b à partir de leur décomposition Ta et Tb. Rappelons que le GCD est le produit des facteurs premiers communs élevés à la puissance minimale.
 - $GCD_decomp(Ta, Tb) = (2 \uparrow 1) * (3 \uparrow 2) = 18.$
5. La fonction « **LCM_decomp** » qui calcule le LCM de nombres a et b à partir de leur décomposition Ta et Tb. Rappelons que le LCM est le produit des facteurs premiers communs élevés à la puissance maximale, et les facteurs premiers non communs.
 - $LCM_decomp(Ta, Tb) = (2 \uparrow 3) * (3 \uparrow 2) * (5 \uparrow 1) * (7 \uparrow 2) * (11 \uparrow 1) = 194040$

EXERCICE 2 (1 + 2 + 2 = 5 POINTS).

Le « code2 » (écrit sur la page 2 de feuille de réponse) permet de

- a. Initialiser un tableau de nombres de 4 chiffres.
- b. Trier ce tableau en utilisant le tri RADIX.
- c. Interclasser le tableau en trois parties : les nombres Bipremier au début du tableau, les nombres pairs à la fin et le reste au milieu.
- d. Afficher le tableau final

Compléter le « code2 » en écrivant en langage C les modules suivants :

4. La fonction « **Biprime** » qui détermine si un nombre est Bipremier. Un BiPremier N est un nombre de 4 chiffres composé de deux nombres de deux chiffres ($N=N_1N_2$, $N_1 \geq 10$ et $N_2 \geq 10$) qui sont tous les deux premiers. Exemples : $N=2313$ ($N_1=23$, $N_2=13$) et $N=6131$ ($N_1=61$, $N_2=31$)
5. La procédure « **Interclassify** » qui place les nombres Bipremier du tableau au début, ensuite les nombres pairs à la fin et le reste au milieu.

6. La procédure « **RADIX** » qui permet d'ordonner un tableau de nombre de 4 chiffres en utilisant le tri radix. Le tri radix permet d'organiser (ordonner) les entiers en examinant leurs chiffres de droite à gauche. Pour un tableau de nombres à 4 chiffres, l'algorithme procède en 4 passes, une pour chaque position (unités, dizaines, centaines, milliers). A chaque passe, il trie le tableau en ordre croissant en se basant sur le principe du tri par sélection : Pour chaque chiffre de 0 à 9, appliquer le tri par sélection en considérant que le minimum est égal au chiffre courant. Ainsi, placer les zéros (s'ils existent) dans les premières cases, puis les uns, et ainsi de suite.

Voici un exemple de déroulement du « code2 » :

Indice	a. Lire	b. Tri Radix				c. Interclasser		d. Afficher
		1 ^{ère} Passe	2 ^{ème} Passe	3 ^{ème} Passe	4 ^{ème} Passe	1 ^{ère} passe	2 ^{ème} passe	
0	2313	1020	2313	1020	1020	2313	2313	2313
1	8345	6131	1020	6131	2313	6131	6131	6131
2	1020	4532	4532	2313	2348	2348	8763	8763
3	6131	2313	6131	8345	4532	4532	8345	8345
4	4532	8763	8345	2348	6131	1020	1020	1020
5	8763	8345	2348	4532	8345	8345	4532	4532
6	2348	2348	8763	8763	8763	8763	2348	2348

EXERCICE 3 (1 + 1 + 2 = 4 POINTS).

Compléter les fonctions du « code3 » (écrit sur la page 4 de feuille de réponse) qui permet de lire une matrice carrée d'ordre n et de vérifier les propriétés suivantes :

1. Diagonale : si tous ses éléments en dehors de la diagonale principale sont nuls.
2. Symétrique : si elle est égale à sa transposée.
3. Magique : si la somme des nombres de chaque ligne, chaque colonne et chaque diagonale est égale à la même constante, appelée constante magique (dans l'exemple ci-dessous, mag_const = 34).

Exemple des matrices d'ordre 4	Diagonale	Symétrique	Magique
	$\begin{pmatrix} 9 & 0 & 0 & 0 \\ 0 & 8 & 0 & 0 \\ 0 & 0 & 5 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$	$\begin{pmatrix} 1 & 3 & 5 & 7 \\ 3 & 2 & 4 & 6 \\ 5 & 4 & 8 & 0 \\ 7 & 6 & 0 & 9 \end{pmatrix}$	$\begin{pmatrix} 1 & 14 & 15 & 4 \\ 12 & 7 & 6 & 9 \\ 8 & 11 & 10 & 5 \\ 13 & 2 & 3 & 16 \end{pmatrix}$

EXERCICE 4 (2 + 1 + 1 = 4 POINTS).

On veut calculer la moyenne du semestre d'un étudiant universitaire. Pour cela, on a besoin de ses informations illustrées dans la figure (dessinée sur la page 5 de la feuille de réponse). En utilisant le langage algorithmique,

1. créer une structure de données permettant de stocker les informations de l'étudiant.
2. écrire la fonction « **avg_module** » permettant de calculer la moyenne d'un module selon la valeur de la variable TCA qui détermine le type du Contrôle Continu (CA) comme indiqué dans le tableau suivant :

TCA	0	1	2	3
Note de CA	CA = 0	CA = TD	CA = TP	CA = (TD + TP)/2
Moyenne	100% Examen	60% Examen + 40% CA		

3. écrire la fonction « **avg_semestre** » permettant de calculer la moyenne du semestre.